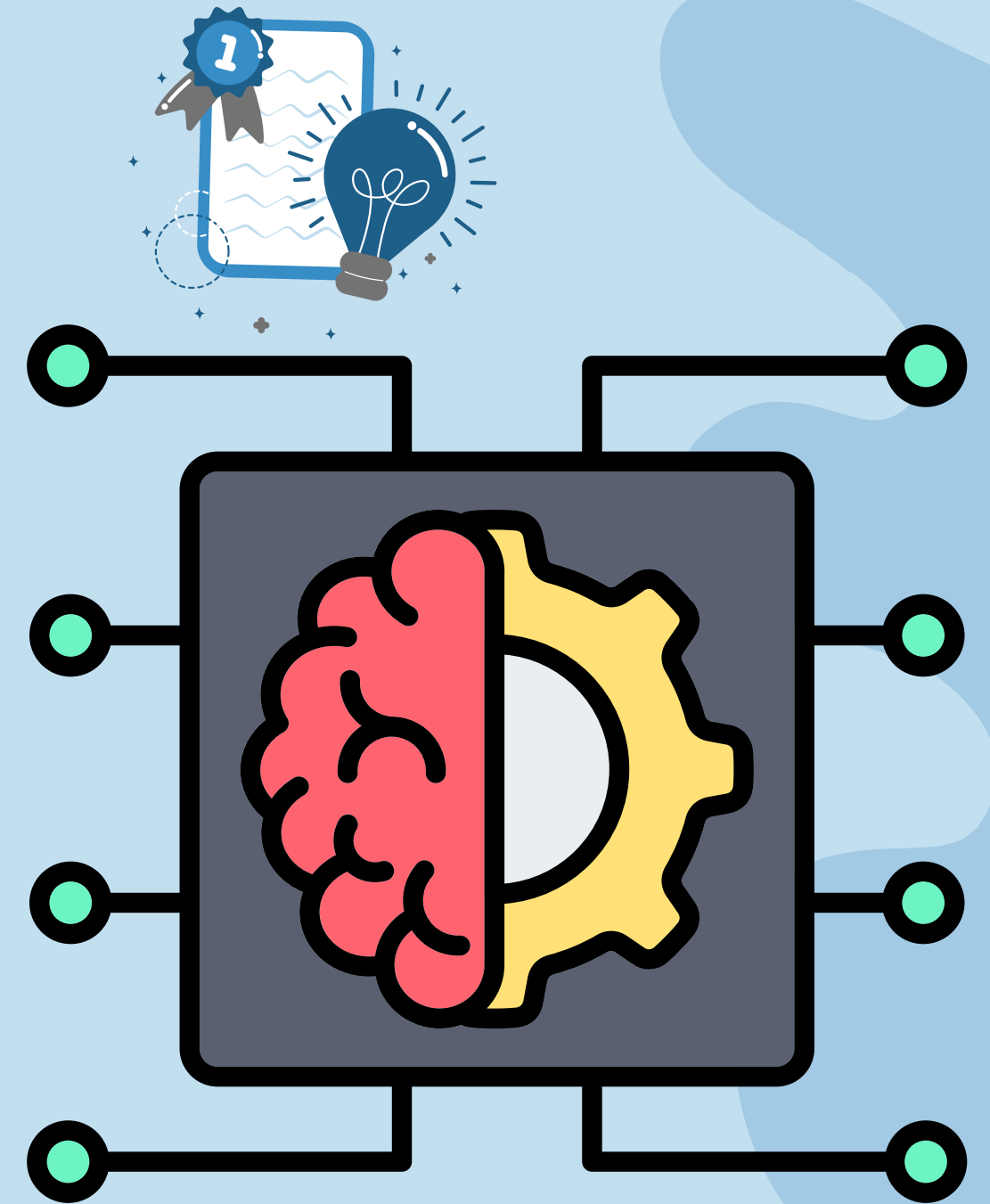
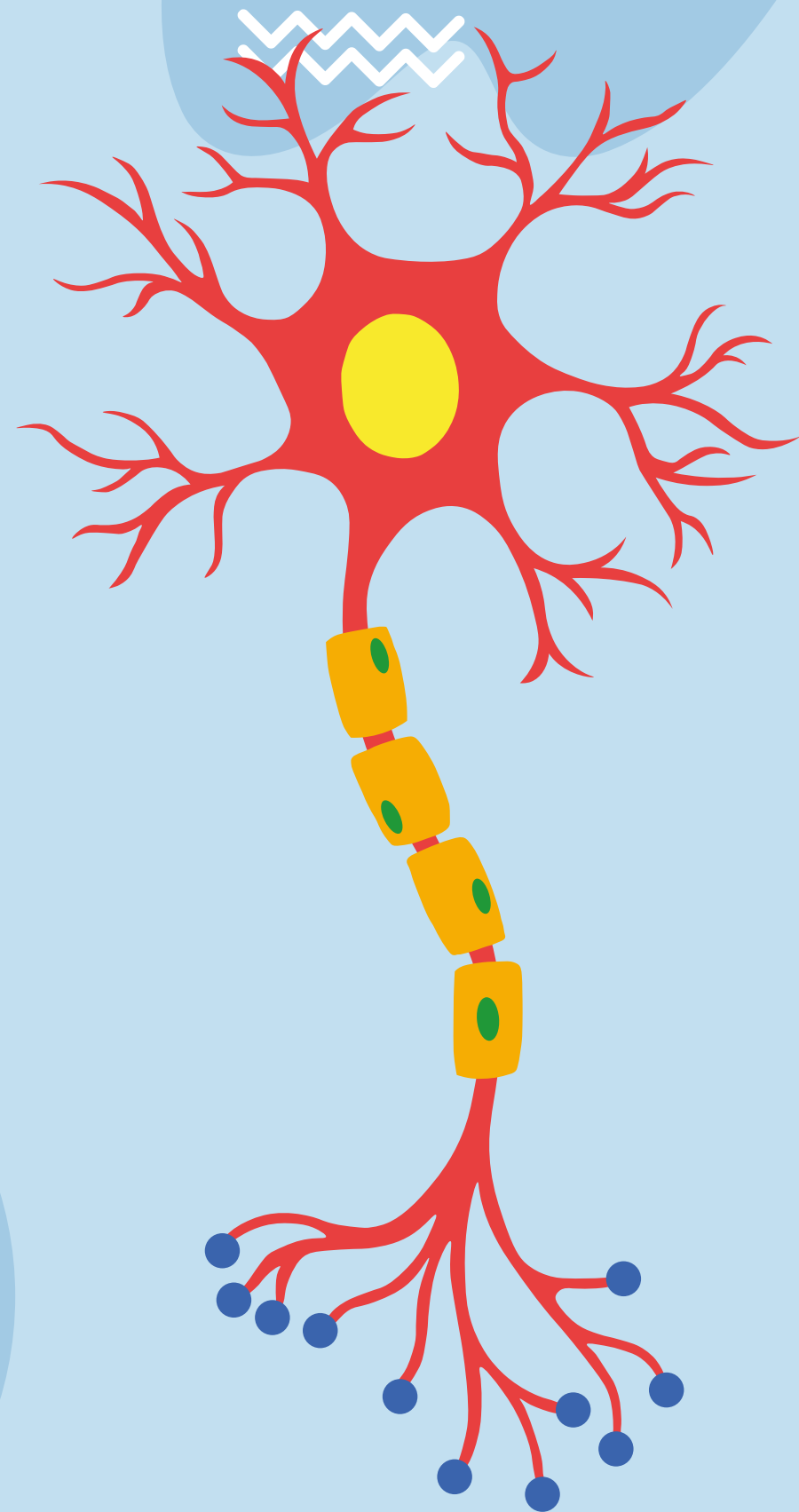


MACHINE LEARNING

Neural Network Activation Functions: A Complete Lecture

Kinza 23K-0842





Introduction

1. Introduction

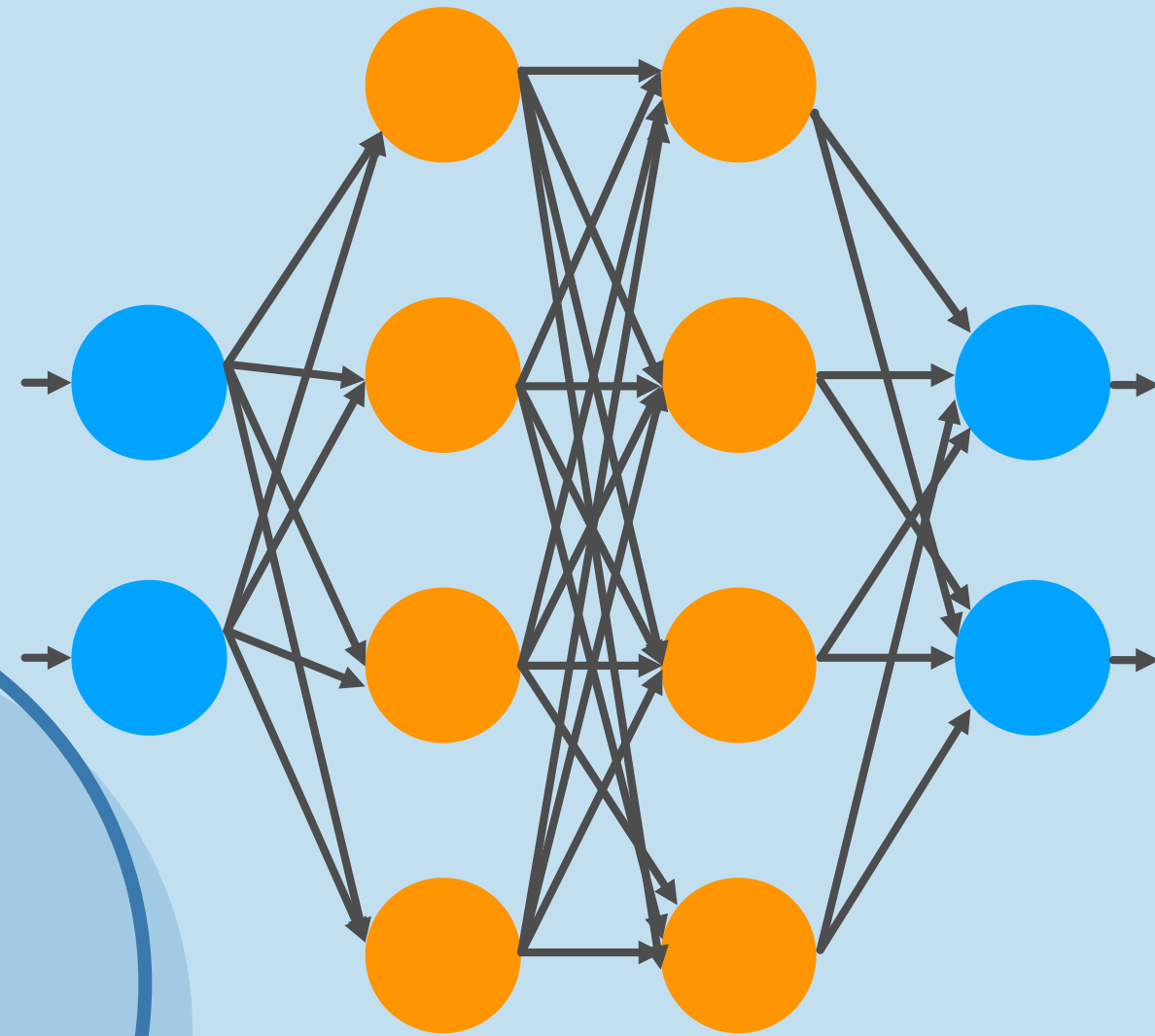
A Neural Network (NN) is a computational model inspired by the human brain. It consists of interconnected nodes (neurons) arranged in layers that learn patterns from data.

Used for:

- Classification (image recognition, spam detection)
- Regression (price prediction)
- NLP (chatbots, sentiment analysis)
- Forecasting



Basic Terminologies

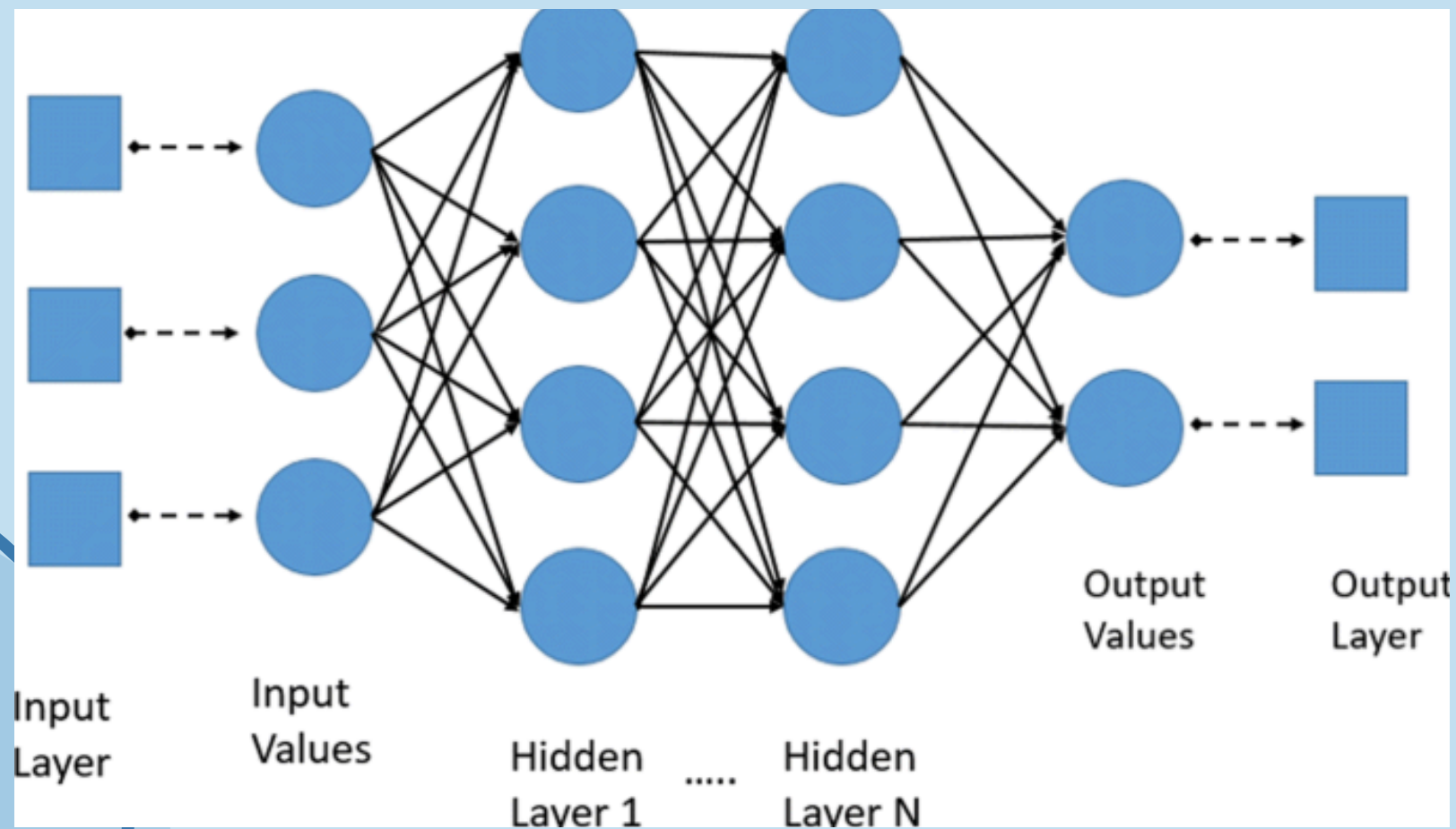


- Neuron / Node: Basic computing unit
- Layer: Input, Hidden, Output
- Weights & Biases: Parameters that guide learning
- Activation Function: Determines output of a neuron
- Loss Function: Measures prediction error
- Learning Rate: Controls the speed of training



Structure of a Neural Network

A typical NN has:



Input Layer – Receives data

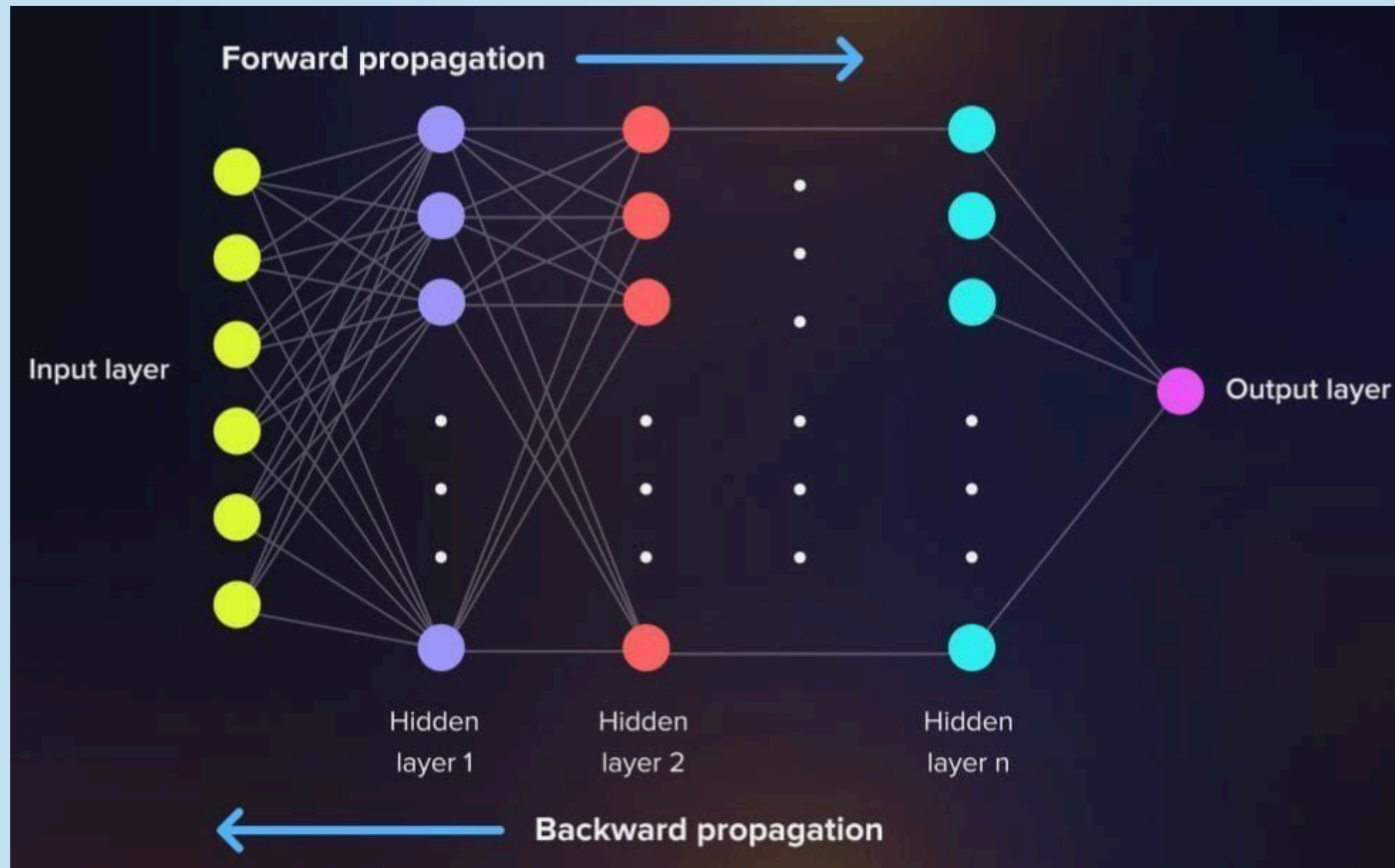


Hidden Layers – Performs computations

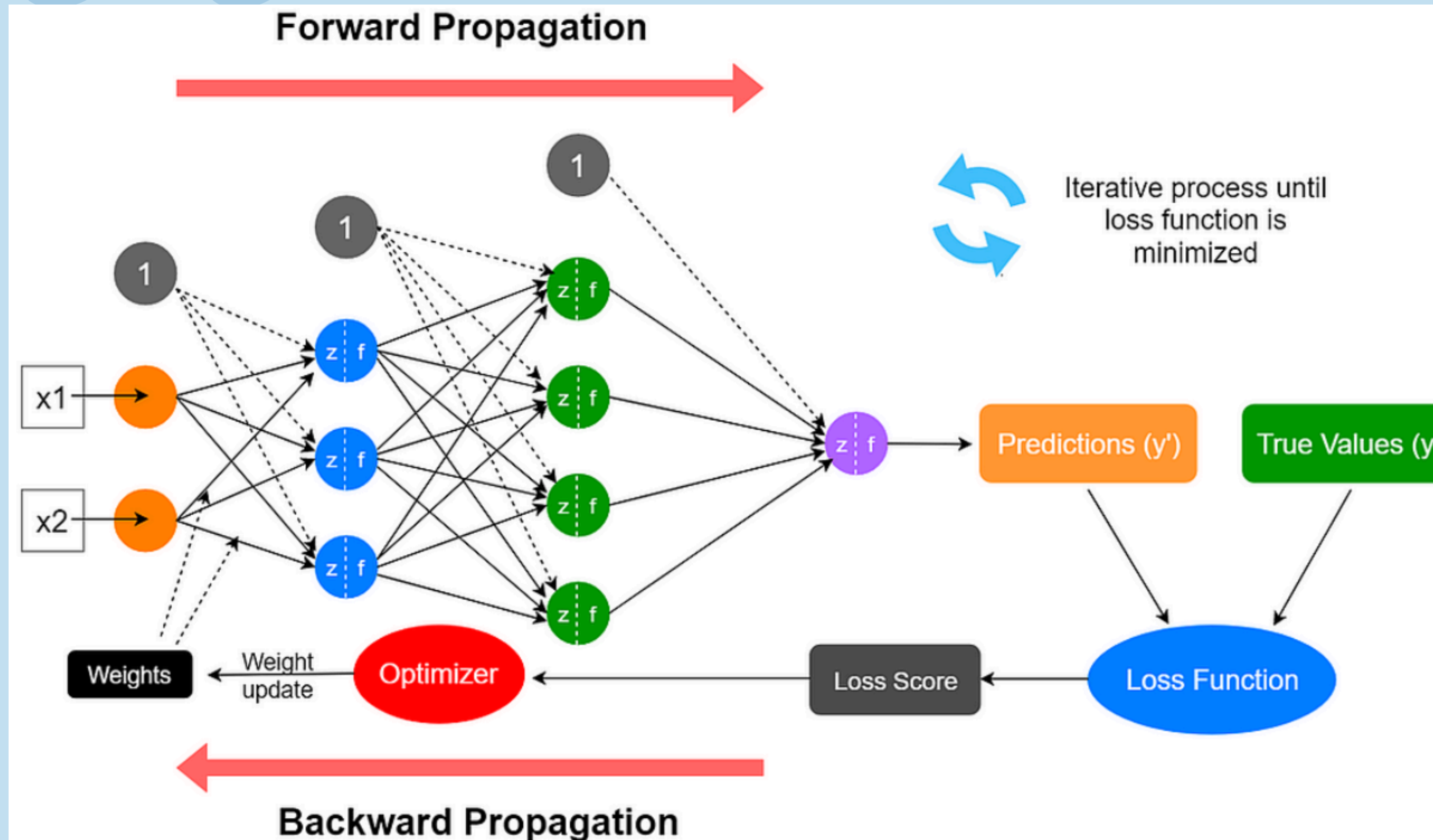


Output Layer – Produces results

How NN works?



How NN works?



How NN works?

Analogy:

- Forward Propagation: Input $\rightarrow$ Weighted Sum $\rightarrow$ Activation $\rightarrow$ Output
- Backward Propagation: Error calculation $\rightarrow$ Weight adjustment
- Forward Propagation: Think of it like “guessing” the answer.
- Backward Propagation: Then “checking the answer” and “adjusting the guess” to improve next time.



Forward Propagation

Purpose: To calculate the output of the neural network from the given input.

Steps:

1. Input: Feed the data into the input layer.
 2. Weighted Sum: Each input is multiplied by a weight and added to a bias:
 3. $z = \sum (x_i \cdot w_i) + b$ where:
 - x_i = input feature
 - w_i = weight for that input
 - b = bias
 4. Activation Function: Apply a function to introduce non-linearity:
 5. $a = f(z)$ Examples of activation functions : Sigmoid, ReLU, Tanh
 6. Output: The result a is passed to the next layer or as final output.
- Summary: Forward propagation “pushes” inputs through the network to generate a prediction.

Backward Propagation

Purpose: To improve the network's predictions by updating weights and biases based on errors.

Steps:

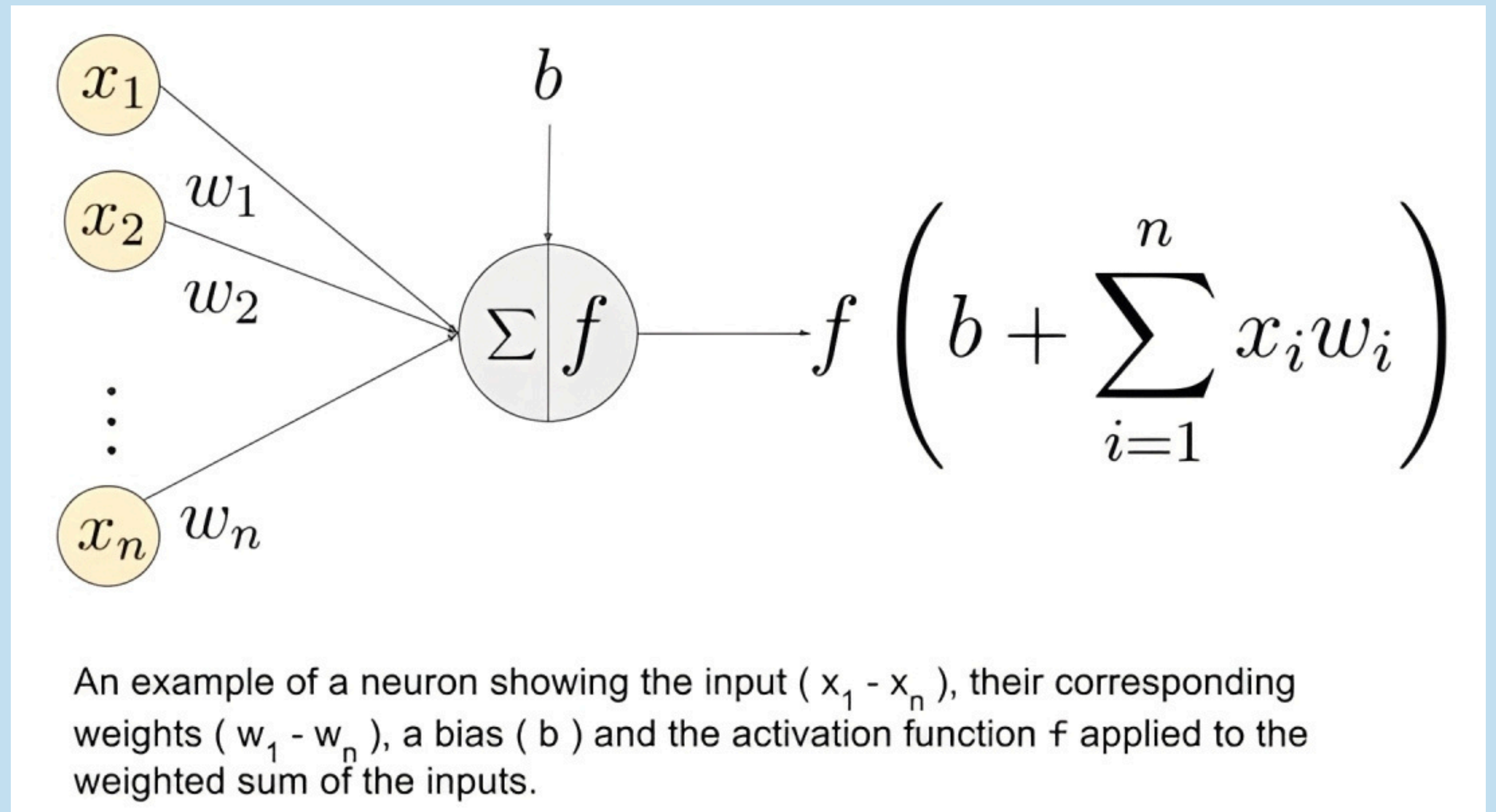
1. Error Calculation: Compare predicted output (y_{pred}) with actual output (y_{true}) using a loss function:
2. Loss = $L(y_{true}, y_{pred})$ Example: Mean Squared Error for regression.
3. Compute Gradients: Determine how much each weight contributed to the error using derivatives (chain rule).
4. Weight Adjustment: Update weights to reduce error using the learning rate (α):
5. $w_{new} = w_{old} - \alpha \cdot \partial L / \partial w$
6. Repeat: This process continues for multiple iterations (epochs) until the network learns to minimize the error.

Summary: Backpropagation is how the network "learns" by correcting mistakes.

Mathematical Formula

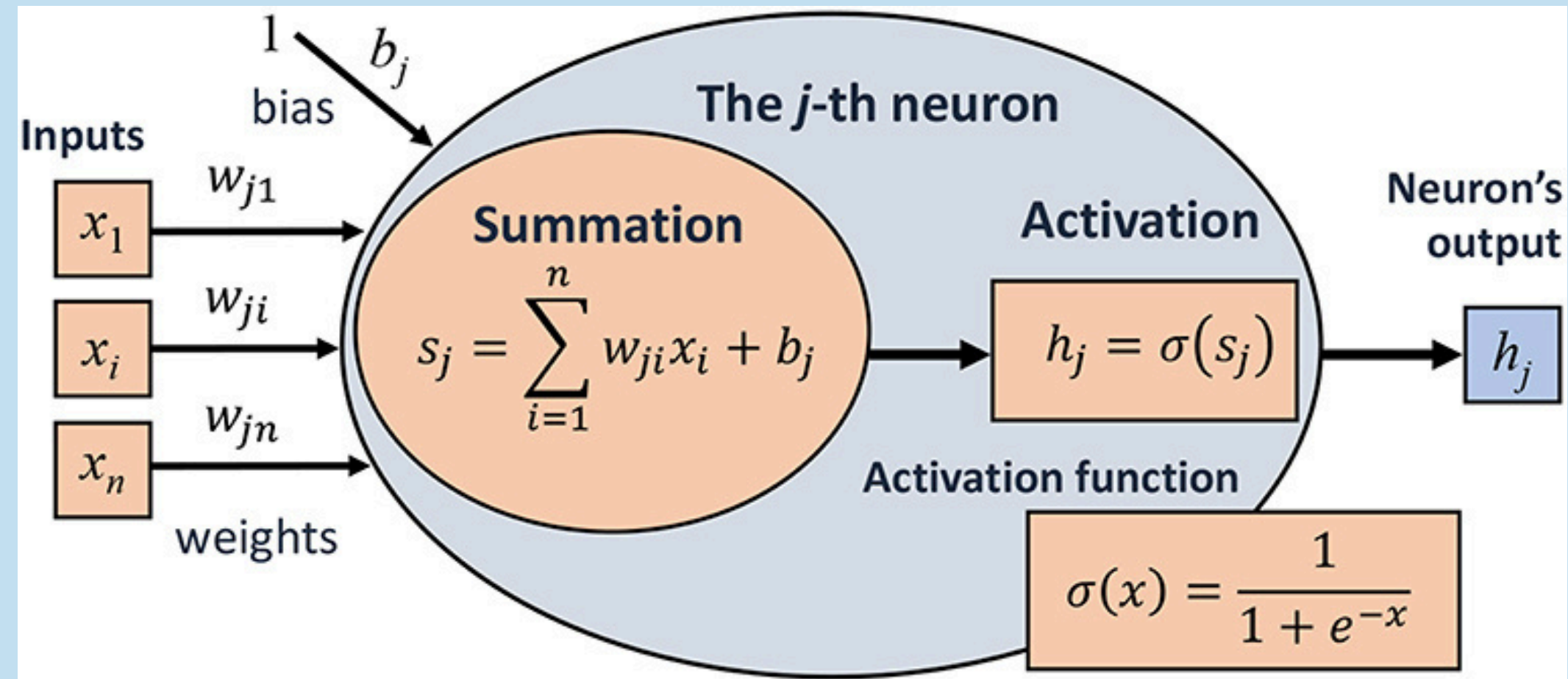
Formula:

- b = bias
- x = input to neuron
- w = weights
- n = the number of inputs from the incoming layer
- i = a counter from 1 to n



Mathematical Formula

- Inputs: $x_1, x_2, \dots, x_n$.
- Weights: $w_{j1}, w_{j2}, \dots, w_{jn}$ — each weight scales the corresponding input.
- Bias: b_j — a trainable offset (acts like an input with fixed value 1).
- Summation (linear combination):
- $s_j = \sum_{i=1}^n w_{ji} x_i + b_j$
- Activation: apply nonlinearity σ to produce output:
- $h_j = \sigma(s_j)$
- Common activations: Sigmoid, ReLU, Tanh, Softmax (for multi-class output).
Activation gives the network ability to learn non-linear mappings.



What are Activation Functions and Why do we need them

Definition & Purpose

- Mathematical functions that map pre-activations (summation) to outputs (output = $g(\text{summation})$)
- Enable nonlinearity, allowing neural networks to learn complex patterns
- Transform weighted sums into meaningful features through hierarchical composition

Critical Properties

- Differentiability for backpropagation and gradient-based optimization
- Prevent deep networks from collapsing into simple linear transformations

Key Insight:

Without activation functions, a neural network with any number of layers would behave exactly like a single-layer linear model, severely limiting its representational capacity.

Linear vs Non-Linear Activation functions

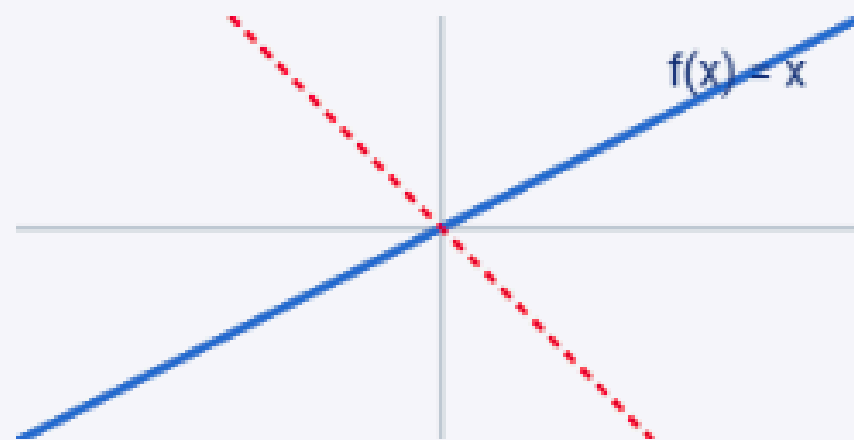
Linear Activation

$$f(x) = x$$

$$f'(x) = 1$$

$$f_{\text{composed}}(x) = W_n W_{n-1} \dots W_1 x + b$$

- › Multiple linear layers collapse to a single affine transformation
- › Cannot model XOR or other nonlinear patterns



Linear networks produce linear decision boundaries

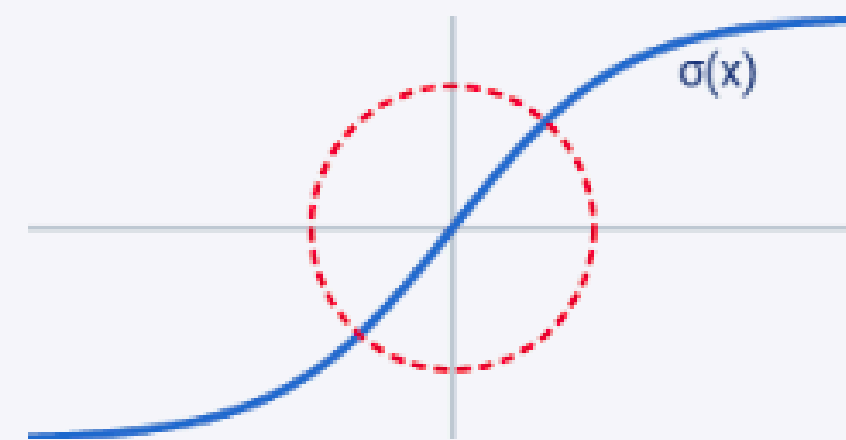
Nonlinear Activation

$$f(x) = \sigma(x), \text{ReLU}(x), \tanh(x), \text{etc.}$$

$$f(g(h(x))) \neq f \circ g \circ h(x)$$

Creates hierarchical feature composition

- › Enables learning of complex patterns through layered abstraction
- › Satisfies universal approximation theorem requirements



Nonlinear networks can form complex decision boundaries

Sigmoid Activation Function

Definition & Purpose

Sigmoid is an S-shaped nonlinear activation function that maps any real value into the range $(0, 1)$. It is used widely for binary classification and probability outputs.

Graph Characteristics

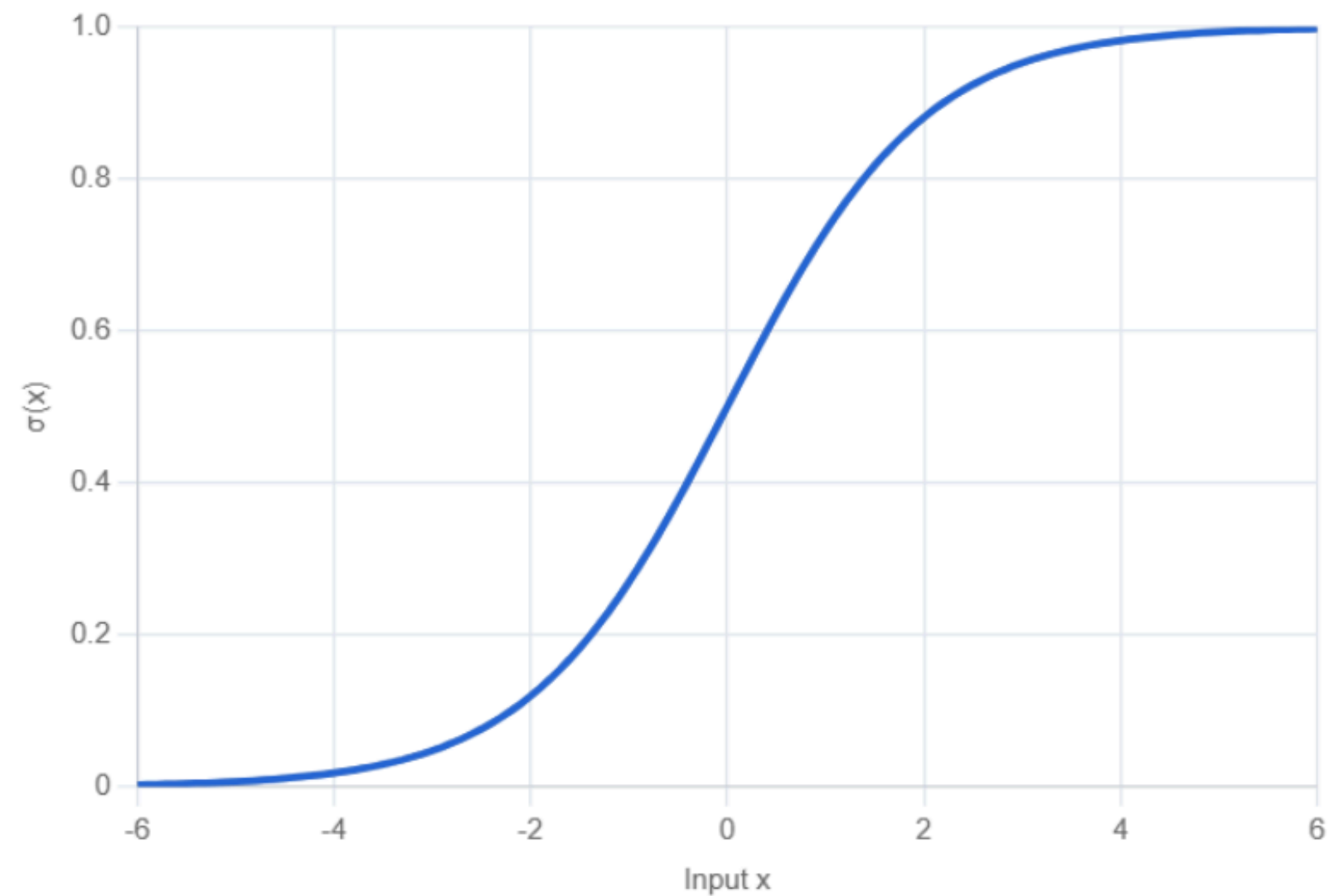
- Smooth S-shaped curve (sigmoid)
- Gradually increases from $0 \rightarrow 1$
- Never touches 0 or 1 (asymptotic)

Properties:

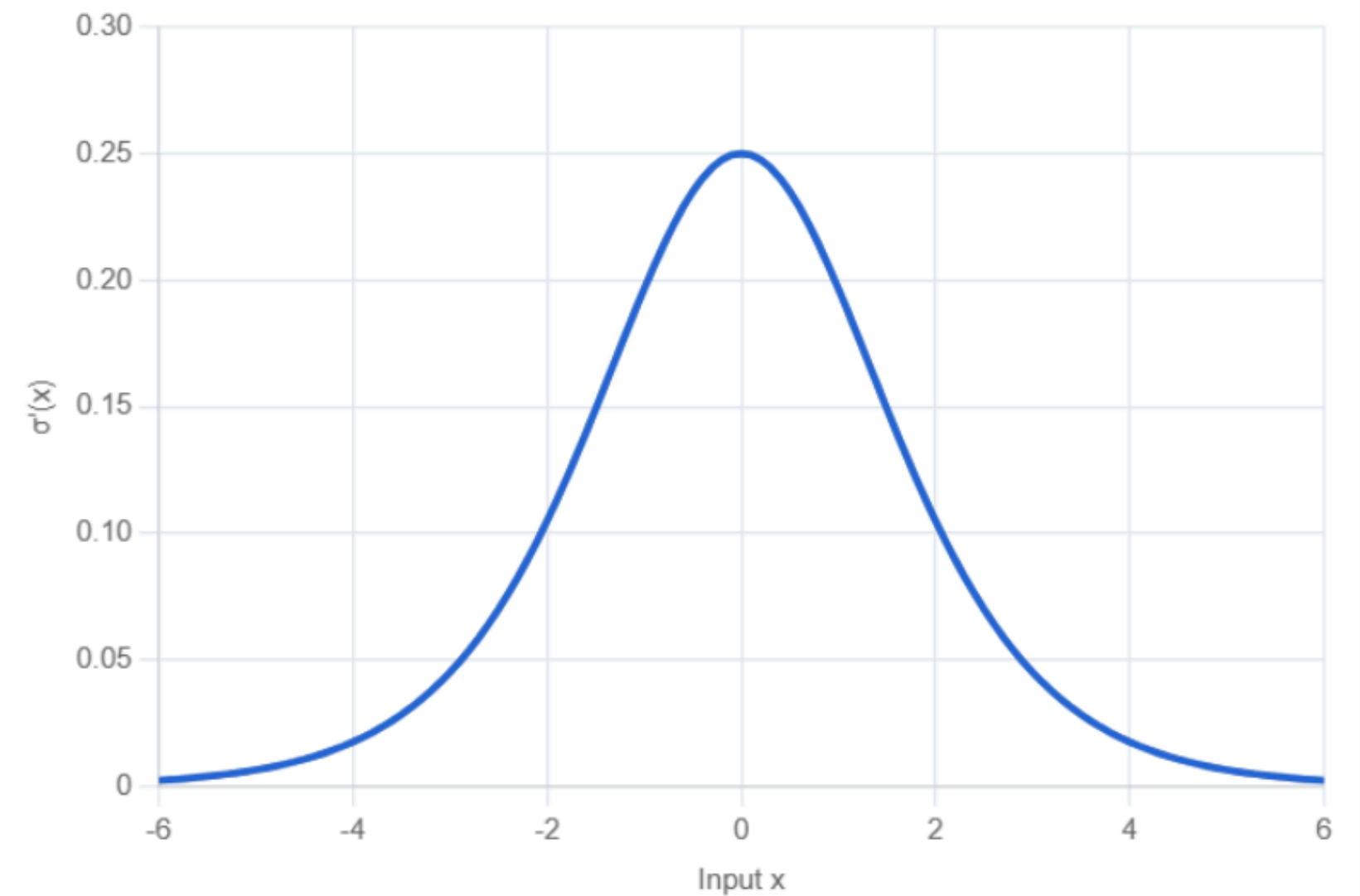
- Range: $(0, 1)$
- Domain: $(-\infty, +\infty)$
- Non-linear: ✓
- Differentiable: ✓ everywhere
- Derivative:
- $f'(x) = f(x)(1-f(x))$
- Centered: (not zero-centered)

Sigmoid Activation Function

Sigmoid Function



Derivative of Sigmoid



Sigmoid Application & Use cases



Where Sigmoid is Used

- > **Binary Classification:** Output layer for binary tasks (values between 0-1)
- > **Gating Mechanisms:** Control information flow in LSTM/GRU networks
- > **Probability Estimation:** Models requiring outputs interpretable as probabilities
- > **Legacy Networks:** Early neural networks before ReLU dominance

Example Architectures



Binary Classifiers: Final layer of neural networks for binary tasks (spam detection, sentiment analysis)



LSTM/GRU Networks: Gate activations in recurrent cells



Autoencoders: Early designs for bounded feature representation



Advantages & Disadvantages



Advantages

- ✓ Smooth, differentiable function with well-defined gradient
- ✓ Output bounded between 0 and 1
- ✓ Natural probabilistic interpretation
- ✓ Clear decision boundary at 0.5



Disadvantages

- ✗ Vanishing gradient for extreme inputs
- ✗ Non-zero-centered output slows training
- ✗ Computationally expensive (exp function)
- ✗ Saturation can lead to slow convergence



Why It Works

- > The bounded output range (0-1) is perfect for binary classification tasks
- > The continuous nature enables smooth gradients where input is near zero
- > Simple derivative formula $\sigma(x)(1-\sigma(x))$ enables efficient backpropagation

Sigmoid Common Problems



Vanishing Gradient Problem

For large positive or negative inputs ($|x| \gg 0$), the gradient of sigmoid becomes extremely small, approaching zero. This causes weights to update very slowly or not at all during backpropagation.

Training Impact: ●●●



Non-Zero Centered Output

Sigmoid outputs are always positive (between 0 and 1), making the gradient updates either all positive or all negative. This causes inefficient zigzagging during optimization in multi-layer networks.

Training Impact: ●●●



Saturation Effects

Neurons can become "saturated" when inputs push activations to the flat regions of sigmoid (near 0 or 1). Once saturated, these neurons effectively stop learning since gradients become near-zero.

Training Impact: ●●●



Initialization Sensitivity

With improper weight initialization (especially large values), neurons can start in saturated regions. This leads to poor gradient flow from the beginning of training and potential network paralysis.

Training Impact: ●●●

Mitigating Strategies:

Use proper weight initialization (e.g., Xavier/Glorot), apply batch normalization to reduce saturation, or replace sigmoid with more modern alternatives like ReLU family functions in hidden layers. For output layers requiring values between 0-1, sigmoid remains appropriate despite these limitations.

TANH Activation Function

Definition & Purpose

Tanh (Hyperbolic Tangent) is a non-linear activation function that maps inputs to a range of $(-1, 1)$.

It is essentially a scaled and shifted Sigmoid, centered around zero.

Graph Characteristics

- S-shaped smooth curve
- Symmetric about the origin
- Outputs negative values for negative x
- Approaches -1 and +1 asymptotically

Mathematical Formula:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

TANH Activation Function

Properties:

- Range: $(-1,1)$
- Domain: $(-\infty, +\infty)$
- Non-linear: ✓
- Differentiable: ✓ everywhere
- Zero-centered: ✓
- Monotonic: ✓ increasing function
- Bounded: ✓
- Saturates: at large $|x| \rightarrow$ causes vanishing gradient
- Derivative:
- $\tanh'(x) = 1 - \tanh^2(x)$

TANH Activation Function

Advantage:

- Zero-centered $\rightarrow$ faster, more stable training
- Strong gradients around 0 $\rightarrow$ better than sigmoid for hidden layers
- Good for handling both negative and positive activations

Disadvantages

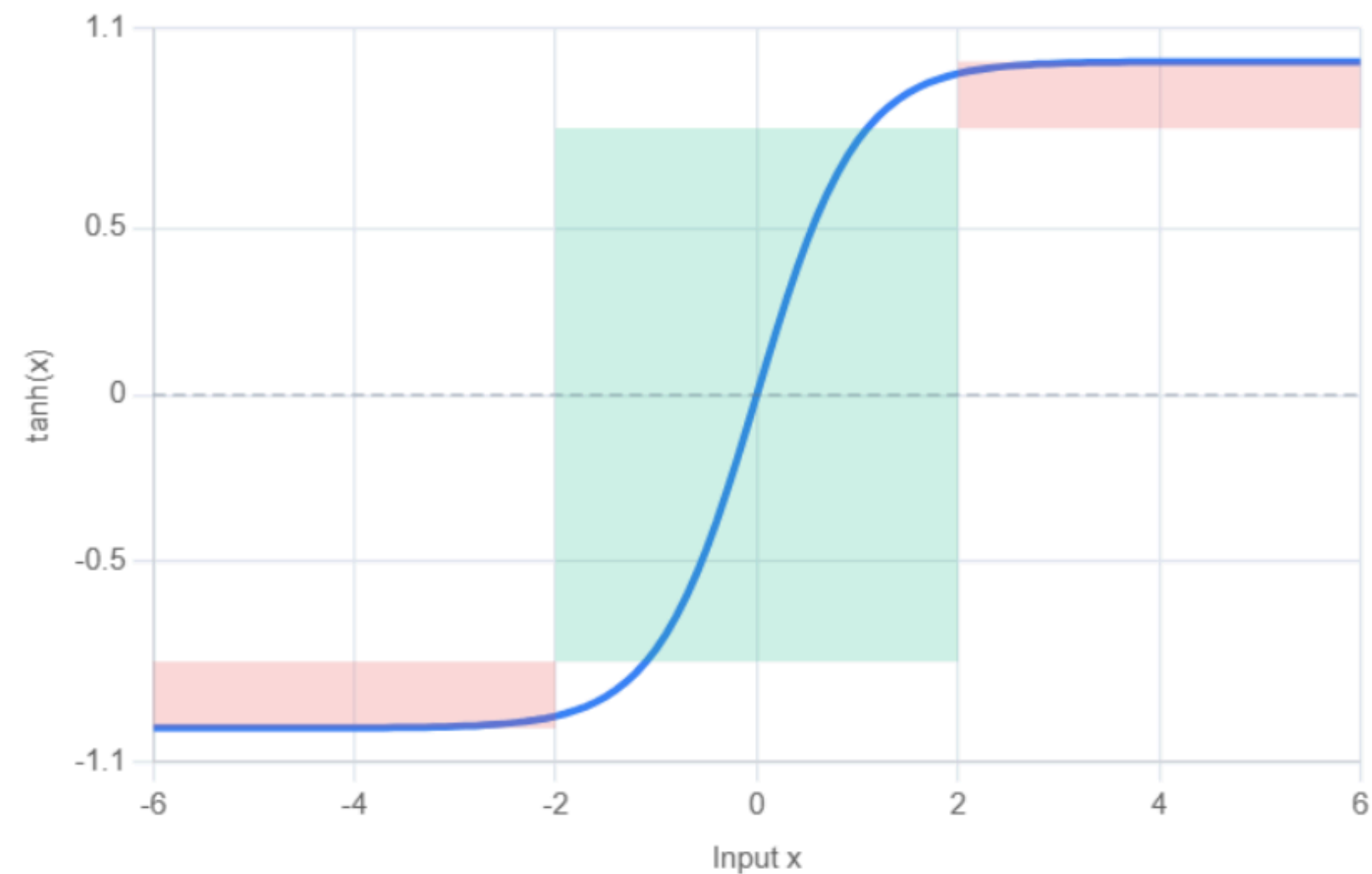
- Still suffers from vanishing gradients for large $|x|$
- Computationally heavier than ReLU (uses exponentials)

Where It Is Used

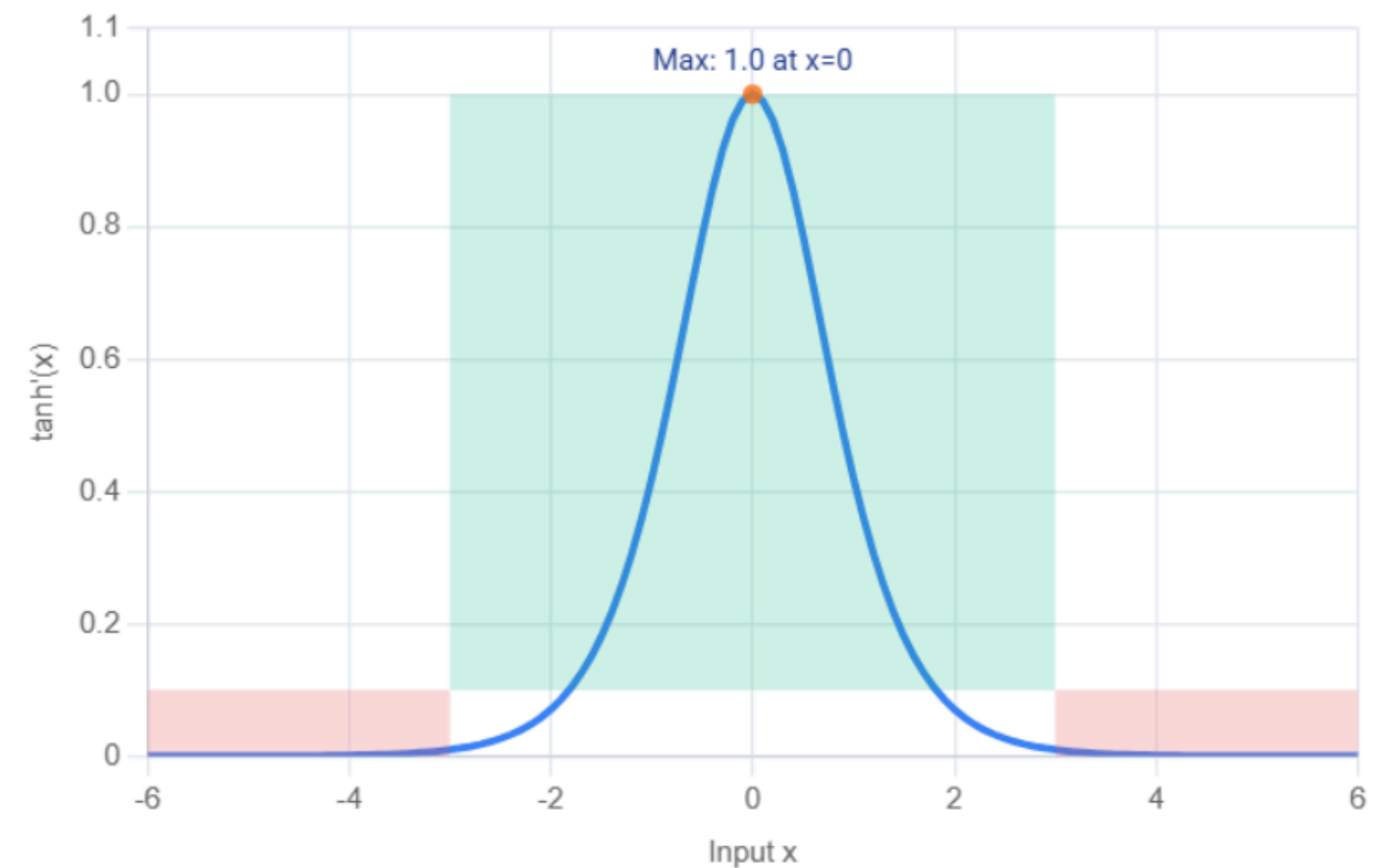
- Hidden layers in neural networks
- RNNs (Recurrent Neural Networks)
- Situations where you want outputs centered around zero

TANH Activation Function

Tanh Function



Derivative of Tanh



■ Function/Derivative ■ Saturation Region (Near-Zero Gradient) ■ Active Region (Significant Gradient)

ReLU (Rectified Linear Unit)

Definition:

ReLU is a non-linear activation function that outputs the input directly if it is positive; otherwise, it outputs 0. It is the most widely used activation in deep neural networks due to its speed and simplicity.

Mathematical formula:

$$\text{ReLU}(x) = \max(0, x)$$

Graph characteristics:

- Flat at 0 for all $x < 0$.
- Straight line with slope 1 for $x \geq 0$
- Simple piecewise linear function
- Most intuitive activation to visualize

ReLU (Rectified Linear Unit)

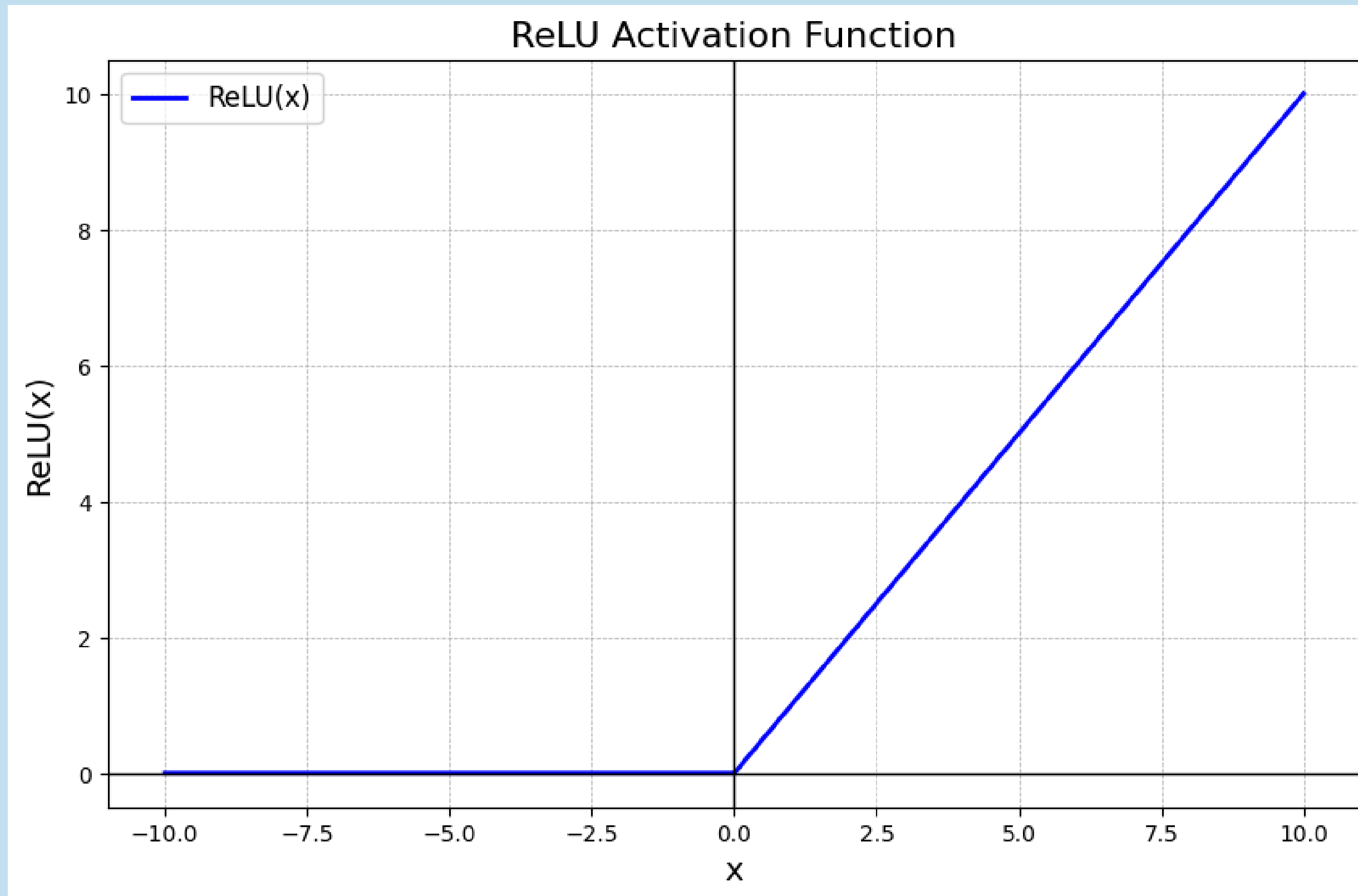
Key Properties

- Range: $[0, +\infty)$
- Domain: $(-\infty, +\infty)$
- Non-linear: ✓ (because of the sharp corner at $x=0$)
- Differentiable: ✓ everywhere except $x=0$
- Zero-centered: No
- Monotonic: ✓
- Unbounded positive side: ✓
- Efficient to compute: ✓

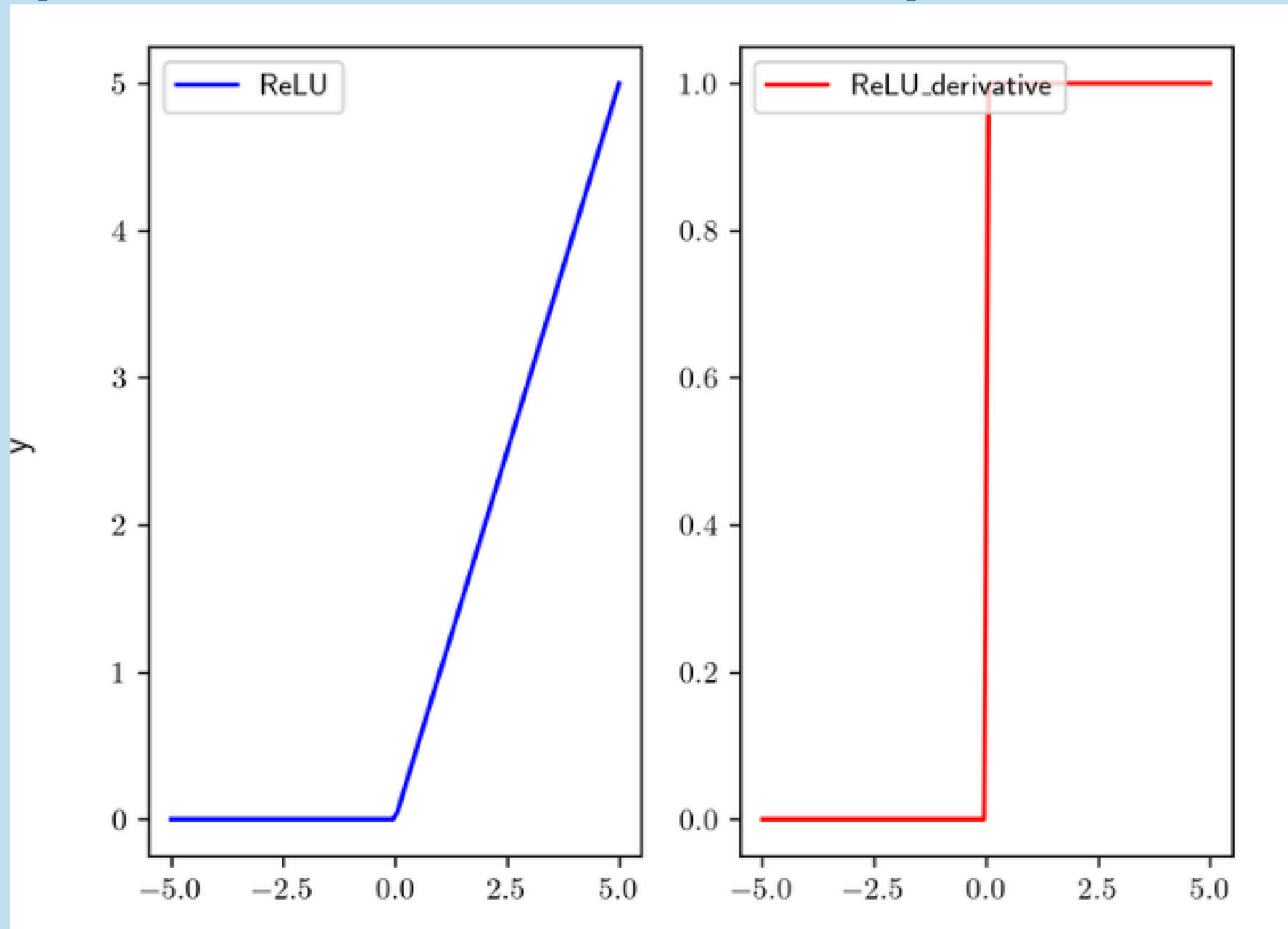
Derivative:

$$\text{ReLU}'(x) = \begin{cases} 0 & x < 0 \\ 1 & x > 0 \end{cases}$$

ReLU (Rectified Linear Unit)



ReLU (Rectified Linear Unit)



Leaky ReLU (Rectified Linear Unit)

Leaky ReLU is a variant of ReLU that allows a small, non-zero slope for negative inputs to avoid the “Dying ReLU” problem.

Mathematical Formula

Typical value: $\alpha=0.01$

$$\text{Leaky ReLU}(x) = \begin{cases} \alpha x, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Graph Characteristics

- Same as ReLU for $x \geq 0$
- Slight downward slope (not flat) for $x < 0$.
- Prevents neurons from becoming inactive.

Leaky ReLU (Rectified Linear Unit)

Key Properties

- Range: $(-\infty, +\infty)$
- Domain: $(-\infty, +\infty)$
- Non-linear: ✓
- Differentiable: ✓ everywhere except at 0
- Monotonic: ✓
- Zero-centered: No
- Unbounded: ✓

Leaky ReLU (Rectified Linear Unit)

Derivative:

$$f'(x) = \begin{cases} \alpha, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

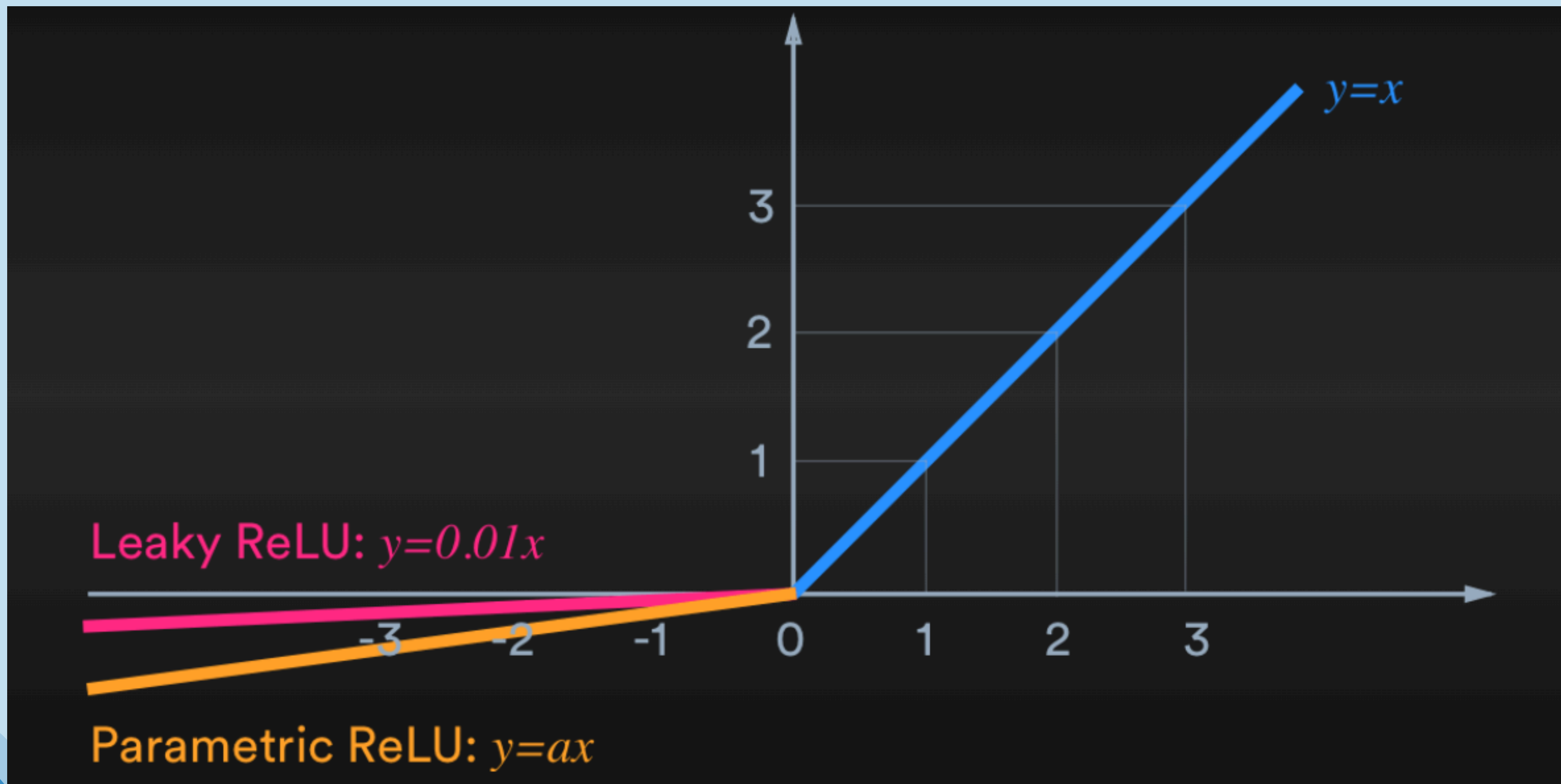
Advantages

- Fixes “Dying ReLU” issue
- Allows gradient flow for negative inputs
- Simple and computationally fast
- Helps model learn features with negative values

Disadvantages

- Output not zero-centered
- Choosing α is arbitrary (too small $\approx$ ReLU)
- Still not ideal for all deep architectures

Leaky ReLU (Rectified Linear Unit)



Summary

Activation	Linear?	Range	Pros	Cons
Linear	Yes	$(-\infty, \infty)$	Simple	No learning of complex patterns
Sigmoid	No	$(0, 1)$	Good for probabilities	Vanishing gradient
Tanh	No	$(-1, 1)$	Zero-centered	Still slow
ReLU	No	$(0, \infty)$	Fast, good for deep nets	Dead neurons
Leaky ReLU	No	$(-\infty, \infty)$	Fixes dead ReLU	Small negative slope

Vanishing & Exploding Gradients

What Are Vanishing & Exploding Gradients?

- During backpropagation, gradients are propagated backward layer by layer.
- If gradients become extremely small → Vanishing Gradient
- If gradients grow extremely large → Exploding Gradient
- Both problems occur due to repeated multiplication of derivatives across many layers.

! Why It Matters?

- Network stops learning (weights barely update).
- Early layers (closer to the input) fail to learn patterns.
- Training becomes extremely slow or unstable.

Loss Function

A loss function measures how well the neural network's predictions match the actual target values.

Smaller loss $\rightarrow$ better model predictions.

Think of it as a "score" for how wrong the model is.

Why It's Important

- Guides the learning process.
- Backpropagation uses the loss to adjust weights.
- Different tasks require different loss functions (e.g., regression vs classification).

Mean Squared Error

used for regression tasks

- y_i = actual value
- $\hat{y}_i$ = predicted value
- n = number of examples

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Explanation:

- Calculates the average of squared differences between actual and predicted values.
- Penalizes large errors more than small errors.

Cross Entropy Loss (Logloss)

where:

- y_i = actual class (0 or 1)
- $\hat{y}_i$ = predicted probability
- n = number of examples

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

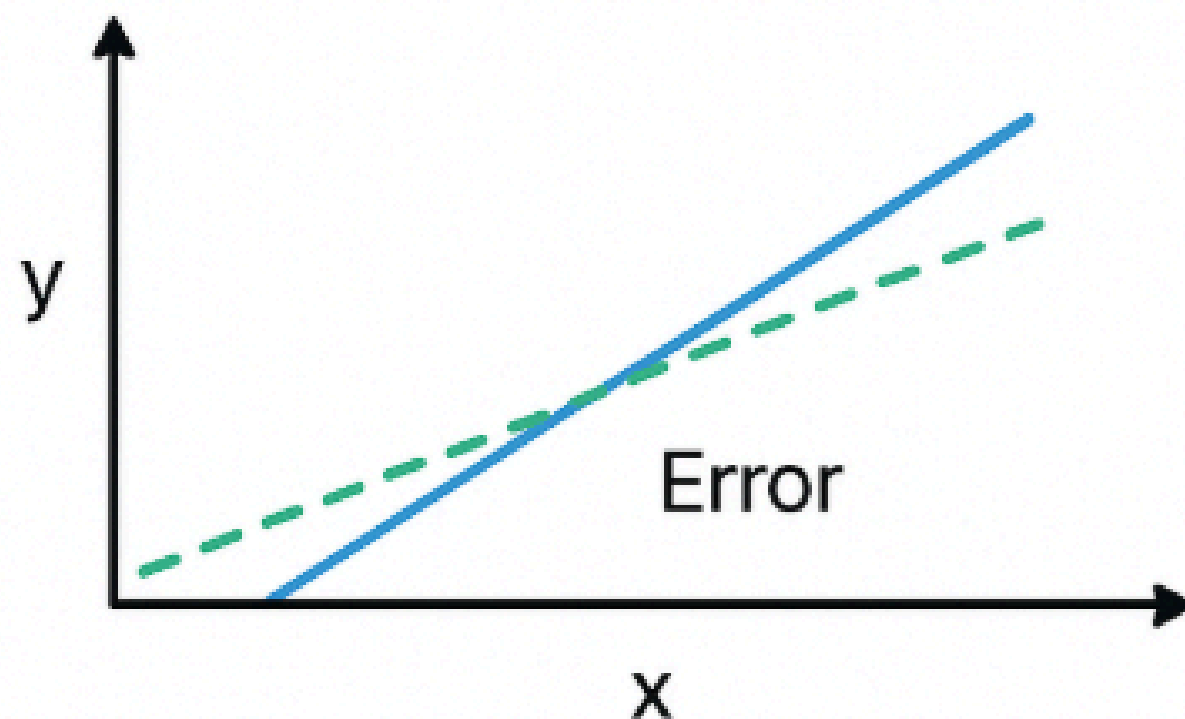
Explanation:

- Measures how well the predicted probabilities match the actual class labels.
- Lower value $\rightarrow$ better predictions.

Loss Functions – Measuring Prediction Error

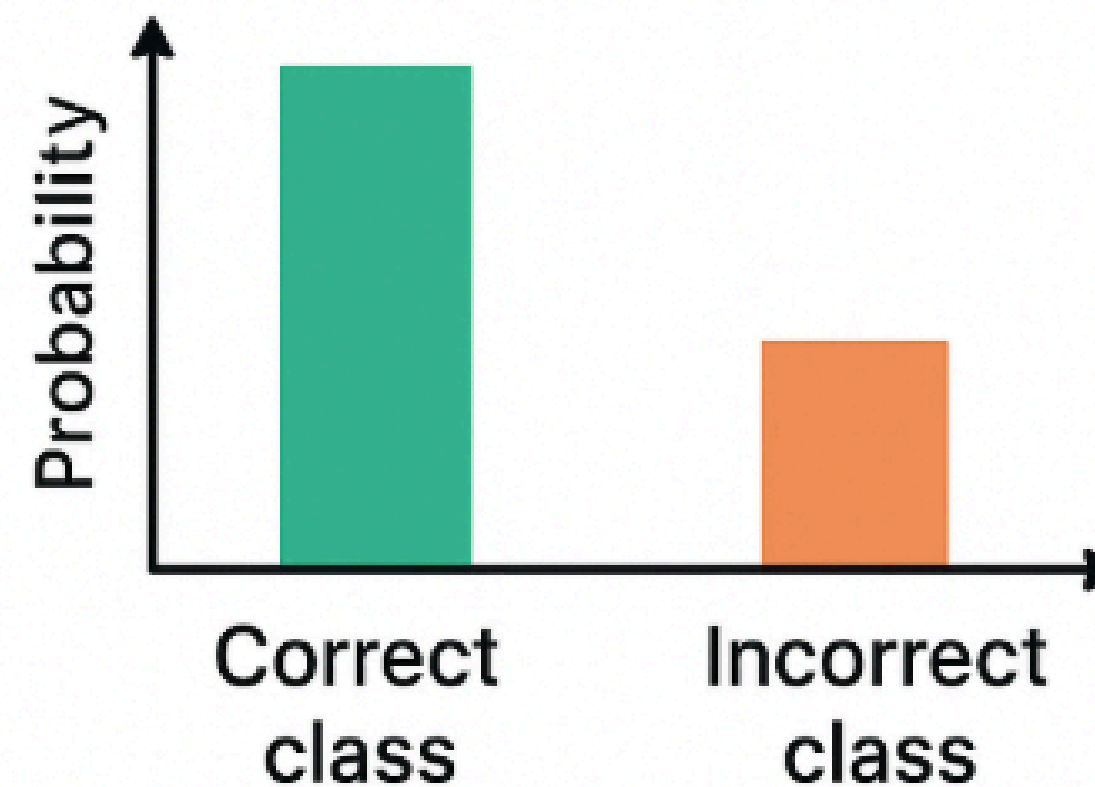
Mean Squared Error
(MSE) – regression

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Cross-Entropy –
classification

$$L = -\frac{1}{n} \sum_{i=1} y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)$$



Summary

- Loss functions are the guide for learning.
- Regression $\rightarrow$ MSE
- Classification $\rightarrow$ Cross-Entropy
- The network updates its weights to minimize the loss during training.



Training a Neural Network

Steps:

1. Initialize weights
2. Forward propagation
3. Compute loss
4. Backpropagation
5. Update weights
6. Repeat for multiple epochs



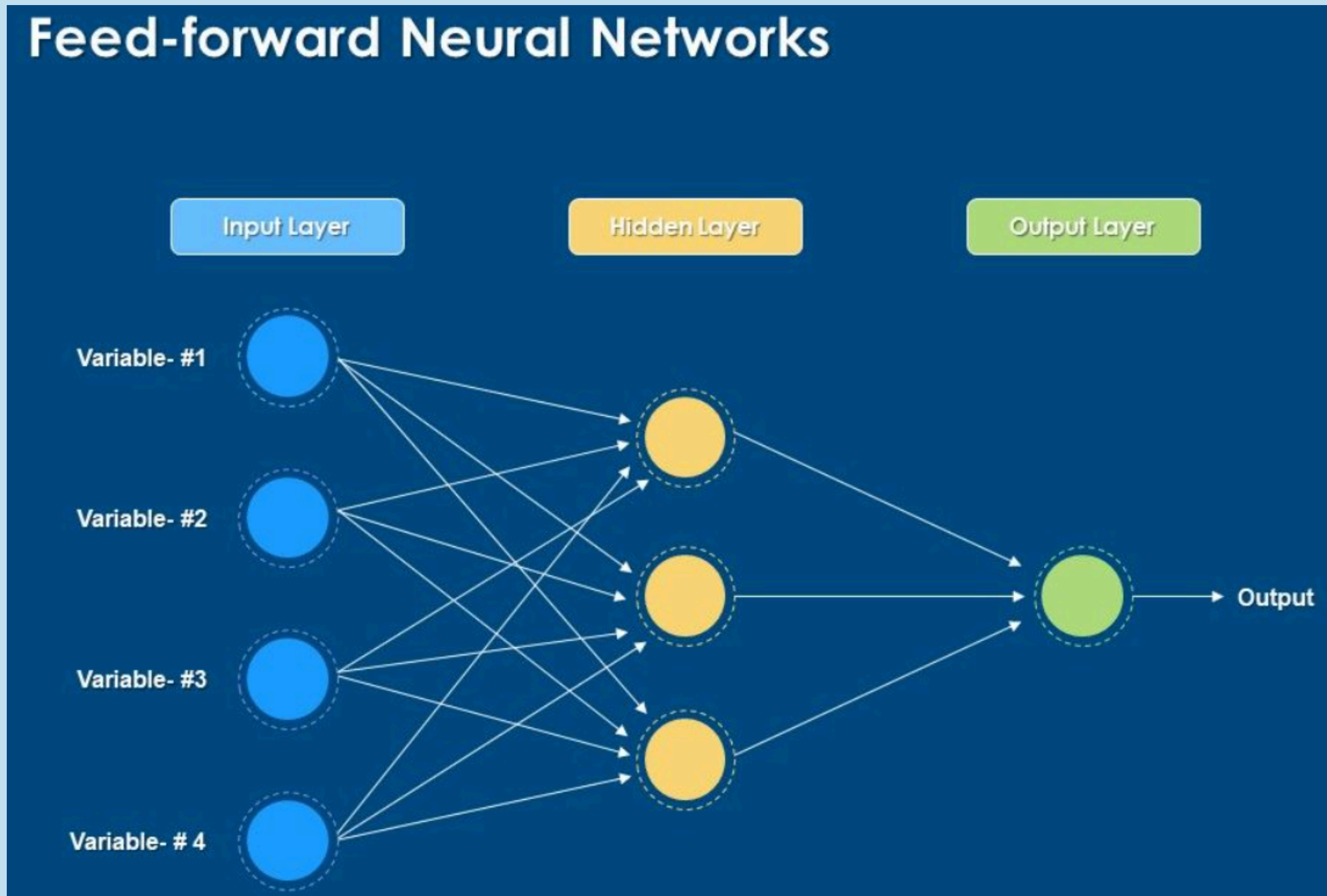
Types of Neural Network

Common Types of Neural Networks

- Feedforward Neural Network (FNN) – Basic type
- Convolutional Neural Network (CNN) – For images
- Recurrent Neural Network (RNN) – For sequences, text, time series

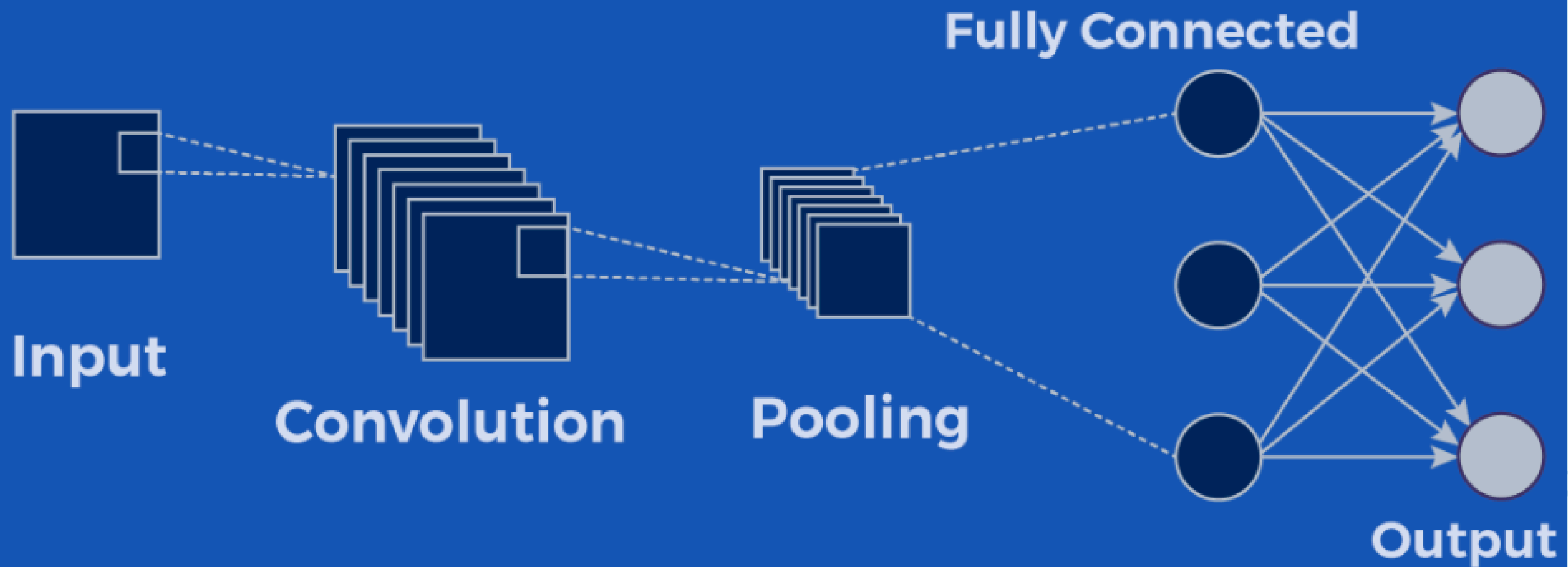


Feedforward Neural Network



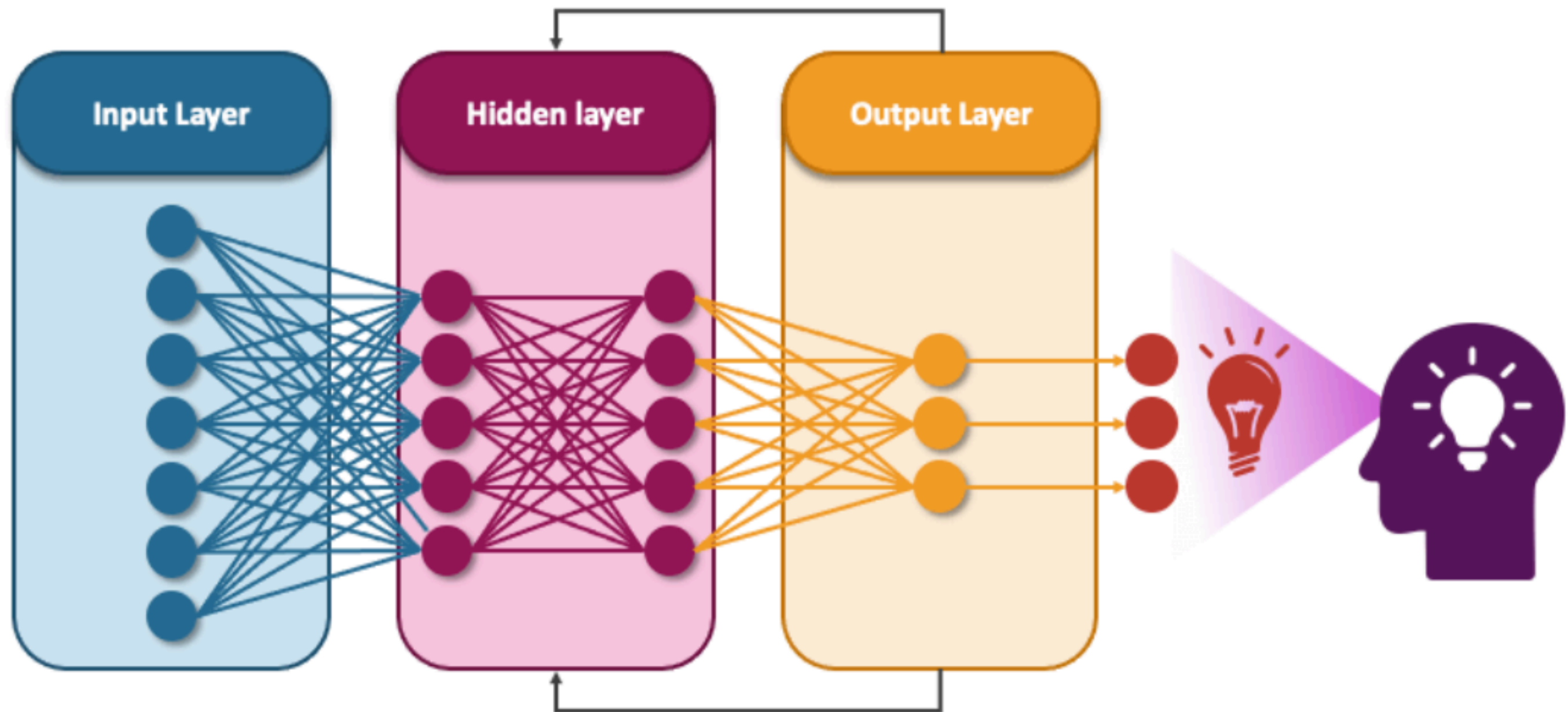
Convolutional Neural Network

Convolutional Neural Networks



Recurrent Neural Network

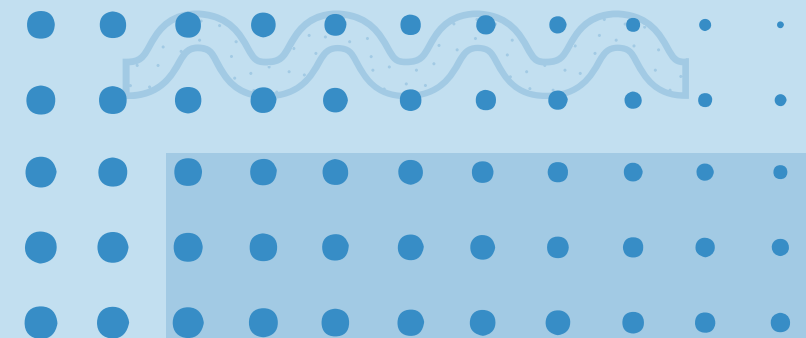
RECURRENT NEURAL NETWORK





Application

- Image recognition (face detection)
- Natural language processing (chatbots, translation)
- Medical diagnosis (cancer detection)
- Self-driving cars (object detection & decision-making)
- Financial forecasting





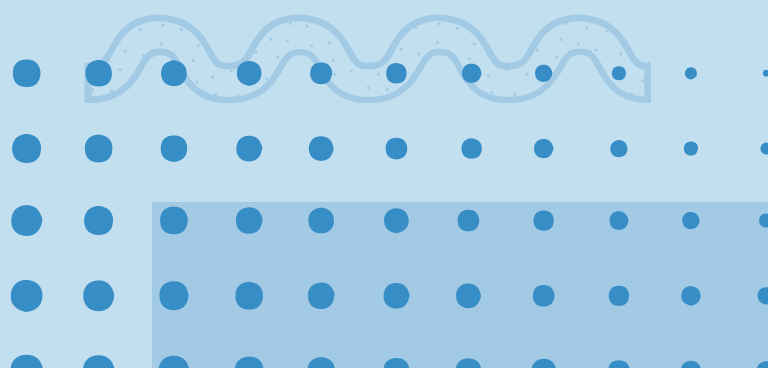
Advantages & Limitations



Advantages:

- Can model complex non-linear relationships
- Learns patterns automatically from data

Limitations:

- Requires large datasets
 - Computationally expensive
 - Black-box nature (hard to interpret)
- 



Summary



- Neural networks are inspired by the human brain
- Work via layers of interconnected neurons
- Learn patterns from data using forward and backward propagation
- Widely used in AI and ML applications

